TRIBHUVAN UNIVERSITY

INSTITUTE OF SCIENCE AND TECHNOLOGY

SOCH COLLEGE OF IT

LAB REPORT OF

“ADVANCE DATABASE”



SUBMITTED TO:

Soch College of IT

Ranipauwa, pokhara

SUBMITTED BY

Nabin Magar (16846)

Semester: 8^{t}

TABLE OF CONTENT :

Lab 1: Review of Basic SQL 1

Lab:2 Relational operators 5

Lab 3: Types of SQL Join: 8

Lab 4: Sub Queries in SQL: 12

Lab 5: To create a procedure, and the query written inside the procedure. Simply call the procedure to execute the query. 15

Lab 6: Pass the parameter to the procedure: IN and OUT parameter 16

Lab 7: Creating and updating Trigger 17

Lab 8: Creating after update trigger 19

Lab 9:

- **a) Creating after update when department is changed** 21
- **b) Creating After Insert Trigger** 22
- **c) After delete of record** 23

Lab 1: Review of Basic SQL

```
mysql> select * from employee;
+--------+----------+-----------+--------+--------+
| Emp_id | Emp_name | Job_title | Salary | Dep_id |
+--------+----------+-----------+--------+--------+
|  78001 | Krishna  | President |  60000 |   1001 |
|  78002 | Binod    | Manager   |  27500 |   3001 |
|  78003 | Cristina | Manager   |  25500 |   1001 |
|  78004 | Jenny    | Manager   |  29500 |   2001 |
|  78005 | Suman    | Analyst   |  31000 |   2001 |
|  78006 | Ajay     | Salesman  |  19000 |   3001 |
|  78007 | Sandesh  | Clerk     |  12000 |   2001 |
+--------+----------+-----------+--------+--------+
```

Tasks:

1. From the following table, write a SQL query to find the salaries of all employee.

Return salary.

2. From the given database table, write a SQL query to count the number of employee with job_title = ‘Manager’.

3. From the following tables,Write a SQL query to find the distinct designations of the employees. Return job name.

4. From the following table, write a SQL query to find those employee who do not belong to the department 2001. Return complete information about the employee.

Answers:

making table named: “employee”

code:

create table employee(Emp_id int, Emp_name varchar(15), Job_title varchar(15), Salary int, Dep_id int);

```
mysql> create table employee(
    -> Emp_id int,
    -> Emp_name varchar(15),
    -> Job_title varchar(15),
    -> Salary int,
    -> Dep_id int);
Query OK, 0 rows affected (0.04 sec)
```

putting the values in table:

code:

insert into employee values (78001,'Krishna','President',60000,1001), (78002,'Binod','Manager',27500,3001), (78003,'Cristina','Manager',25500,1001), (78004,'Jenny','Manager',29500,2001), (78005,'Suman','Analyst',31000,2001), (78006,'Ajay','Salesman',19000,3001), (78007,'Sandesh','Clerk',12000,2001);

```
mysql> insert into employee values
    -> (78001,'Krishna','President',60000,1001),
    -> (78002,'Binod','Manager',27500,3001),
    -> (78003,'Cristina','Manager',25500,1001),
    -> (78004,'Jenny','Manager',29500,2001),
    -> (78005,'Suman','Analyst',31000,2001),
    -> (78006,'Ajay','Salesman',19000,3001),
    -> (78007,'Sandesh','Clerk',12000,2001);
Query OK, 7 rows affected (0.01 sec)
Records: 7  Duplicates: 0  Warnings: 0
```

1) From the following table, write a SQL query to find the salaries of all employees.

Return salary.

Ans:

Select Emp_name,Salary from employee;

```
mysql> Select Emp_name,Salary from employee;
+----------+--------+
| Emp_name | Salary |
+----------+--------+
| Krishna  |  60000 |
| Binod    |  27500 |
| Cristina |  25500 |
| Jenny    |  29500 |
| Suman    |  31000 |
| Ajay     |  19000 |
| Sandesh  |  12000 |
+----------+--------+
7 rows in set (0.00 sec)
```

2) From the given database table, write a SQL query to count the number of employees with job_title = ‘Manager’.

Ans:

select count(Job_title) from employee where Job_title='Manager';

```
mysql> select count(Job_title) from employee where Job_title='Manager';
+------------------+
| count(Job_title) |
+------------------+
|                3 |
+------------------+
1 row in set (0.00 sec)
```

3) From the following tables, Write a SQL query to find the distinct designations of the employees. Return job name.

Ans:

select distinct(Job_title) from employee;

```
mysql> select distinct(Job_title) from employee;
+-----------+
| Job_title |
+-----------+
| President |
| Manager   |
| Analyst   |
| Salesman  |
| Clerk     |
+-----------+
5 rows in set (0.00 sec)
```

4) From the following table, write a SQL query to find those employees who do not belongs to the department 2001. Return complete information about the employees.

Ans:

select * from employee where Dep_id!=2001;

```
mysql> select * from employee where Dep_id!=2001;
+--------+----------+-----------+--------+--------+
| Emp_id | Emp_name | Job_title | Salary | Dep_id |
+--------+----------+-----------+--------+--------+
|  78001 | Krishna  | President |  60000 |   1001 |
|  78002 | Binod    | Manager   |  27500 |   3001 |
|  78003 | Cristina | Manager   |  25500 |   1001 |
|  78006 | Ajay     | Salesman  |  19000 |   3001 |
+--------+----------+-----------+--------+--------+
4 rows in set (0.00 sec)
```

Lab:2 Relational operators

```
mysql> select * from market;
+-------------+---------------+-----------+-------+-------------+
| Customer_id | Customer_name | City      | Sales | Salesman_id |
+-------------+---------------+-----------+-------+-------------+
|        3002 | Nikesh        | Kathmandu |   500 |        5001 |
|        3007 | Binod         | Kathmandu |  2500 |        5001 |
|        3005 | Ganesh        | Pokhara   |  2000 |        5002 |
|        3008 | Julie         | Butwal    |  3000 |        5002 |
|        3004 | Fadindra      | Pokhara   |  2500 |        5006 |
|        3009 | Grishma       | Palpa     |  1000 |        5003 |
+-------------+---------------+-----------+-------+-------------+
```

Tasks:

1. From the following table, write a SQL query to find the details of the customers who have sales above 1000.

2. From the following table, write a SQL query to find all the customers in ‘Kathmandu’ city who have a sales value above 1000.

3. From the following table, write a SQL query to find all the customers who have a sales value between 2000 and 3000.

4. From the following table, write a SQL query to find all the customers in ‘Kathmandu’ or ‘Pokhara’ city .

5. From the following tables, write a SQL query to find all customers who are not from ‘Kathmandu’ city.

Answers:

creating tables name: Market

code:

create table market(Customer_id int, Customer_name varchar(15), City varchar(15), Sales int, Salesman_id int);

```
mysql> create table market(
    -> Customer_id int,
    -> Customer_name varchar(15),
    -> City varchar(15),
    -> Sales int,
    -> Salesman_id int);
Query OK, 0 rows affected (0.04 sec)
```

Entering the values:

code:

insert into market values (3002,'Nikesh','Kathmandu',500,5001), (3007,'Binod','Kathmandu',2500,5001), (3005,'Ganesh','Pokhara',2000,5002), (3008,'Julie','Butwal',3000,5002), (3004,'Fadindra','Pokhara',2500,5006), (3009,'Grishma','Palpa',1000,5003);

```
mysql> insert into market values
    -> (3002,'Nikesh','Kathmandu',500,5001),
    -> (3007,'Binod','Kathmandu',2500,5001),
    -> (3005,'Ganesh','Pokhara',2000,5002),
    -> (3008,'Julie','Butwal',3000,5002),
    -> (3004,'Fadindra','Pokhara',2500,5006),
    -> (3009,'Grishma','Palpa',1000,5003);
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

1) From the following table, write a SQL query to find the details of the customers who have sales above 1000.

ans:

select * from market where Sales>1000;

```
mysql> select * from market where Sales>1000;
+-------------+---------------+-----------+-------+-------------+
| Customer_id | Customer_name | City      | Sales | Salesman_id |
+-------------+---------------+-----------+-------+-------------+
|        3007 | Binod         | Kathmandu |  2500 |        5001 |
|        3005 | Ganesh        | Pokhara   |  2000 |        5002 |
|        3008 | Julie         | Butwal    |  3000 |        5002 |
|        3004 | Fadindra      | Pokhara   |  2500 |        5006 |
+-------------+---------------+-----------+-------+-------------+
4 rows in set (0.00 sec)
```

2) From the following table, write a SQL query to find all the customers in ‘Kathmandu’ city who have a sales value above 1000.

ans:

select * from market where City = 'Kathmandu' and Sales>1000;

```
mysql> select * from market where City = 'Kathmandu' and Sales>1000
+-------------+---------------+-----------+-------+-------------+
| Customer_id | Customer_name | City      | Sales | Salesman_id |
+-------------+---------------+-----------+-------+-------------+
|        3007 | Binod         | Kathmandu |  2500 |        5001 |
+-------------+---------------+-----------+-------+-------------+
1 row in set (0.00 sec)
```

3) From the following table, write a SQL query to find all the customers who have a sales value between 2000 and 3000.

ans:

select * from market where Sales<3000 and Sales>2000;

```
mysql> select * from market where Sales<3000 and Sales>2000;
+-------------+---------------+-----------+-------+-------------+
| Customer_id | Customer_name | City      | Sales | Salesman_id |
+-------------+---------------+-----------+-------+-------------+
|        3007 | Binod         | Kathmandu |  2500 |        5001 |
|        3004 | Fadindra      | Pokhara   |  2500 |        5006 |
+-------------+---------------+-----------+-------+-------------+
2 rows in set (0.00 sec)
```

4) From the following table, write a SQL query to find all the customers in ‘Kathmandu’ or ‘Pokhara’ city .

ans:

select * from market where City = 'Kathmandu' or City ='Pokhara';

```
mysql> select * from market where City = 'Kathmandu' or  City ='Pokhara';
+-------------+---------------+-----------+-------+-------------+
| Customer_id | Customer_name | City      | Sales | Salesman_id |
+-------------+---------------+-----------+-------+-------------+
|        3002 | Nikesh        | Kathmandu |   500 |        5001 |
|        3007 | Binod         | Kathmandu |  2500 |        5001 |
|        3005 | Ganesh        | Pokhara   |  2000 |        5002 |
|        3004 | Fadindra      | Pokhara   |  2500 |        5006 |
+-------------+---------------+-----------+-------+-------------+
4 rows in set (0.00 sec)
```

5) From the following tables, write a SQL query to find all customers who are not from ‘Kathmandu’ city.

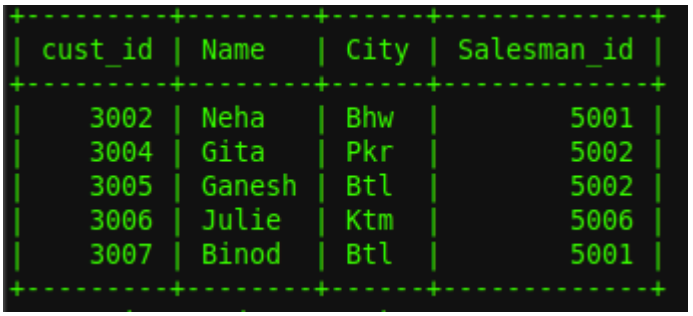
ans:

select * from market where City!= 'Kathmandu';

```
mysql> select * from market where City!= 'Kathmandu';
+-------------+---------------+---------+-------+-------------+
| Customer_id | Customer_name | City    | Sales | Salesman_id |
+-------------+---------------+---------+-------+-------------+
|        3005 | Ganesh        | Pokhara |  2000 |        5002 |
|        3008 | Julie         | Butwal  |  3000 |        5002 |
|        3004 | Fadindra      | Pokhara |  2500 |        5006 |
|        3009 | Grishma       | Palpa   |  1000 |        5003 |
+-------------+---------------+---------+-------+-------------+
4 rows in set (0.01 sec)
```

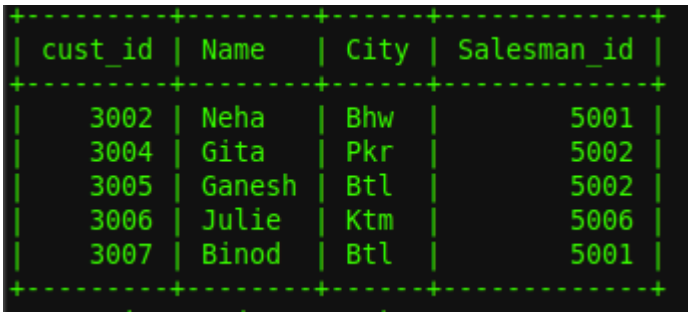
Lab 3: Types of SQL Join:

Table: customer (from lab 3)



cust_id	Name	City	Salesman_id
3002	Neha	Bhw	5001
3004	Gita	Pkr	5002
3005	Ganesh	Btl	5002
3006	Julie	Ktm	5006
3007	Binod	Btl	5001

Table: salesman (from lab 3)



cust_id	Name	City	Salesman_id
3002	Neha	Bhw	5001
3004	Gita	Pkr	5002
3005	Ganesh	Btl	5002
3006	Julie	Ktm	5006
3007	Binod	Btl	5001

tasks:

1. Find the salesmen and customers with their name and cities, who belongs to the same city.

2. Prepare a list for which salesman are working for which customer and commissions earned by the salesman.

3. Prepare a Cartesian Product between salesman and customer i.e each salesman will appear for all customers.

Answers:

1) Find the salesmen and customers with their name and cities, who belongs to the same city.

ans:

select customer.Name,customer.City,salesman.Name from customer inner join salesman where customer.City = salesman.City;

```
mysql> select customer.Name,customer.City,salesman.Name from custo
mer inner join salesman where customer.City = salesman.City;
+--------+------+---------+
| Name   | City | Name    |
+--------+------+---------+
| Gita   | Pkr  | John    |
| Julie  | Ktm  | Nikesh  |
| Binod  | Btl  | Prakash |
| Ganesh | Btl  | Prakash |
| Julie  | Ktm  | Mahesh  |
| Gita   | Pkr  | Priya   |
+--------+------+---------+
6 rows in set (0.00 sec)
```

2) Prepare a list for which salesman are working for which customer and commissions earned by the salesman.

ans:

select salesman.Name,salesman.Commission,customer.Name from salesman inner join customer where salesman.Salesman_id = customer.Salesman_id;

```
+--------+------------+--------+
| Name   | Commission | Name   |
+--------+------------+--------+
| John   |       1500 | Neha   |
| Nikesh |       1300 | Gita   |
| Nikesh |       1300 | Ganesh |
| Mahesh |       1400 | Julie  |
| John   |       1500 | Binod  |
+--------+------------+--------+
5 rows in set (0.00 sec)
```

3) Prepare a Cartesian Product between salesman and customer i.e each salesman will appear for all customers.

Ans:

select customer.Name, salesman.Name from customer inner join salesman;

```
mysql> select customer.Name, salesman.Name from customer inner join salesman;
+--------+---------+
| Name   | Name    |
+--------+---------+
| Binod  | John    |
| Julie  | John    |
| Ganesh | John    |
| Gita   | John    |
| Neha   | John    |
| Binod  | Nikesh  |
| Julie  | Nikesh  |
| Ganesh | Nikesh  |
| Gita   | Nikesh  |
| Neha   | Nikesh  |
| Binod  | Prakash |
| Julie  | Prakash |
| Ganesh | Prakash |
| Gita   | Prakash |
| Neha   | Prakash |
| Binod  | Mahesh  |
| Julie  | Mahesh  |
| Ganesh | Mahesh  |
| Gita   | Mahesh  |
| Neha   | Mahesh  |
| Binod  | Priya   |
| Julie  | Priya   |
| Ganesh | Priya   |
| Gita   | Priya   |
| Neha   | Priya   |
+--------+---------+
25 rows in set (0.00 sec)
```

Lab 4: Sub Queries in SQL:

Table 1: **salesman(from lab 3)**

Salesman_id	Name	City	Commission
5001	John	Pkr	1500
5002	Nikesh	Ktm	1300
5005	Prakash	Btl	1100
5006	Mahesh	Ktm	1400
5007	Priya	Pkr	1300

Table 2: **orders**

Ord_no	Amount	Cust_id	Salesman_id
7001	15000	3005	5002
7002	6500	3007	5001
7004	11000	3004	5002
7005	2400	3006	5006
7009	27000	3002	5005

Table 3: **customer (from lab 3)**

cust_id	Name	City	Salesman_id
3002	Neha	Bhw	5001
3004	Gita	Pkr	5002
3005	Ganesh	Btl	5002
3006	Julie	Ktm	5006
3007	Binod	Btl	5001

Tasks:

1. Displays all the records of customer who are served by the salesman named ‘Nikesh’.

2. Displays all the orders issued by salesman name ‘John’.

3. Displays all the name of customers whose sales amount is above 10000.

4. Displays the name of all salesman who have provided sales amount above average.

Select name from salesman where salesman_id = (select salesman_id from orders where Amount>(Select avg(amount) from orders))

5. Displays the commission amount of salesman working in ‘Btl’.

Answers:

1) Displays all the records of customer who are served by the salesman named ‘Nikesh’.

ans:

select * from customer where Salesman_id = (select Salesman_id from salesman where Name = 'Nikesh');

```
mysql> select * from customer where Salesman_id =
    -> (select Salesman_id from salesman where Name = 'Nikesh');
+---------+--------+------+-------------+
| cust_id | Name   | City | Salesman_id |
+---------+--------+------+-------------+
|    3004 | Gita   | Pkr  |        5002 |
|    3005 | Ganesh | Btl  |        5002 |
+---------+--------+------+-------------+
2 rows in set (0.00 sec)
```

2) Displays all the orders issued by salesman name ‘John’.

ans:

select * from orders where Salesman_id = (select Salesman_id from salesman where Name='John');

```
mysql> select * from orders where Salesman_id = (select Salesman_id from salesman where Name='John');
+--------+--------+---------+-------------+
| Ord_no | Amount | Cust_id | Salesman_id |
+--------+--------+---------+-------------+
|   7002 |   6500 |    3007 |        5001 |
+--------+--------+---------+-------------+
1 row in set (0.00 sec)
```

3) Displays all the name of customers whose sales amount is above 10000.

ans:

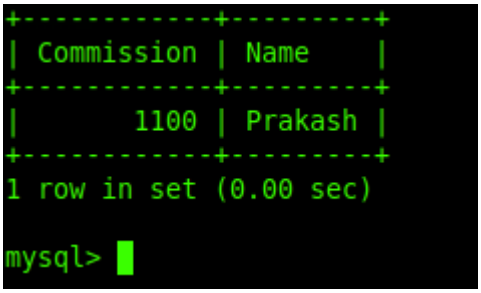
select Name from customer where cust_id in (select Cust_id from orders where Amount >10000);

```
mysql> select *  from customer where cust_id = (select Cust_id from orders where Amount>10000);
ERROR 1242 (21000): Subquery returns more than 1 row
mysql> select Name from customer where cust_id in (select Cust_id from orders where Amount>10000);
+--------+
| Name   |
+--------+
| Ganesh |
| Gita   |
| Neha   |
+--------+
3 rows in set (0.00 sec)
```

5) Displays the commission amount of salesman working in ‘Btl’.

ans:

select Commission,Name from salesman where City= 'Btl';



Lab 5: To create a procedure, and the query written inside the procedure. Simply call the procedure to execute the query.

Create the following table : **employee**

emp_id	name	address	salary
101	James	pkr	15000
102	Philip	pkr	20000
103	Kiara	ktm	20000

SQL :

Creating a table named employee

- Create table employee (emp_id int, name varchar(50), salary int);
- Insert into employee
- Values(101, "James","pkr", 15000) , (102, "Philip","pkr", 20000) ,
- (103, "Kiara","ktm", 20000);

Making the procedure

- delimiter //
  create procedure getalldata()
  begin
  select * from employee ;
  end //
- delimiter ;

Accessing the data

- call getalldata() ;

Output:

```
mysql> call getalldata() ;
+--------+--------+---------+--------+
| emp_id | name   | address | salary |
+--------+--------+---------+--------+
|    101 | James  | pkr     |  15000 |
|    102 | Philip | pkr     |  20000 |
|    103 | Kiara  | ktm     |  20000 |
+--------+--------+---------+--------+
3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Lab 6: Pass the parameter to the procedure: IN and OUT parameter

SQL for IN parameter :

- delimiter //
  create procedure getbyaddress (in city varchar(50))
  begin
  select * from employee
  where address = city ;
  end //
- delimiter ;
- call getbyaddress(‘pkr’) ;

Output:

```
mysql> call getbyaddress("pkr") ;
+--------+--------+---------+--------+
| emp_id | name   | address | salary |
+--------+--------+---------+--------+
|    101 | James  | pkr     |  15000 |
|    102 | Philip | pkr     |  20000 |
+--------+--------+---------+--------+
2 rows in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

SQL for OUT parameter:

- delimiter //
  create procedure sal_city(in city varchar (50) , out total int)
  begin
  select sum(salary) into total
  from employee
  where address = city ;
  end //
- delimiter ;
- call sal_city(‘pkr’,@total);
- select @total;

Output:

```
mysql> select @total;
+--------+
| @total |
+--------+
|  35000 |
+--------+
```

Lab 7: Creating and updating Trigger

Table 1: Create the table “customer”.

acc_no	cname	balance
101	Ram	7000
102	Shyam	12000

Table 2 : Create a new empty table “Statement”

acc_no	balance

SQL:

- Create table customer (acc_no int primary key , cname varchar(50), balance int) ;
- Insert into customer
- values (101 , "Ram", 7000), (102 , "Shyam", 12000);
- Create table statement (acc_no int , balance int, foreign key (acc_no) references
- customer(acc_no));

Now creating a trigger to insert old values into the statement record before updating any record on the employee table .

- Delimiter //
  create trigger update_cus
  before update on customer
  for each row
  begin
  insert into statement
  values (old.acc_no , old.balance) ;
  end //
- Delimiter ;

Now making updates to invoke trigger,

- Update customer set balance = balance + 3000 where acc_no = 101;
- Update customer set balance = balance + 3000 where acc_no = 101;
- Update customer set balance = balance + 5000 where acc_no = 102 ;
- Select * from statement ;
- Select * from customer ;

Output:

```
mysql> Select * from statement ;
+--------+---------+
| acc_no | balance |
+--------+---------+
|    101 |    7000 |
|    101 |   10000 |
|    102 |   12000 |
+--------+---------+
3 rows in set (0.00 sec)

mysql> Select * from customer ;
+--------+-------+---------+
| acc_no | cname | balance |
+--------+-------+---------+
|    101 | Ram   |   13000 |
|    102 | Shyam |   17000 |
+--------+-------+---------+
2 rows in set (0.00 sec)
```

Lab 8: Creating after update trigger

Create two tables: employee and department

Table: employee

Eid	Name	Salary	Dno
1	Juliet	15000	21
2	Romeo	17000	21
3	Joseph	20000	31

Table: department

Dname	Dno	Tot_sal
Account	21	32000
HR	31	20000

SQL:

- Create table employee (eid int , name varchar(50), salary int, dno int) ;
- Insert into employee
- values (1, "Juliet", 15000, 21) , (2, "Romeo", 17000, 21) , (3, "Joseph", 20000, 31);

- Create table department (dname varchar(50), dno int, tot_sal int) ;
- Insert into department
- values ("Account" , 21 , 32000) , ("HR" , 31, 20000);

- Delimiter //
  Create trigger after_up
  after update on employee
  for each row
  begin
  update department set tot_sal = tot_sal + new.salary - old.salary
  where dno = new.dno ;
  end //
- Delimiter ;
- Update employee set salary = salary + 3000 where name="Juliet" ;
- Select * from department ;

```
mysql> Select * from department ;
+---------+------+---------+
| dname   | dno  | tot_sal |
+---------+------+---------+
| Account |   21 |   35000 |
| HR      |   31 |   20000 |
+---------+------+---------+
2 rows in set (0.00 sec)
```

Lab 9:

a) **Creating after update when department is changed**
b) **Creating After Insert Trigger**
c) **After delete of record**

a) Creating after update when department is changed

SQL:

- Delimiter //
- Create trigger new_dep
  after update on department
  for each row
  begin
  update department
  set tot_sal = tot_sal + new.tot_sal where dno = new.dno ;
  update department
  set tot_sal = tot_sal - old.tot_sal where dno = old.dno ;
  end //
- Delimiter ;
- Update employee set dno = dno + 10 where name ="Romeo" ;
- Select * from employee;

Output:

```
mysql> select * from employee;
+------+--------+--------+------+
| eid  | name   | salary | dno  |
+------+--------+--------+------+
|    1 | Juliet |  18000 |   21 |
|    2 | Romeo  |  17000 |   31 |
|    3 | Joseph |  20000 |   31 |
+------+--------+--------+------+
3 rows in set (0.00 sec)
```

b) Creating After Insert Trigger

We will use the same table as above

SQL :

- Delimiter //
- create trigger after_in
  after insert on employee
  for each row
  begin
  update department set tot_sal = tot_sal + new.salary
  where dno = new.dno ;
  end //
- Delimiter ;

Now inserting a new employee to invoke a trigger,

- Insert into employee values ('Paul' , 18000, 31) ;
- Select * from department ;

Output:

Previous

Table Employee:

```
mysql> Select * from employee ;
+------+--------+--------+------+
| eid  | name   | salary | dno  |
+------+--------+--------+------+
|    1 | Juliet |  18000 |   21 |
|    2 | Romeo  |  17000 |   31 |
|    3 | Joseph |  20000 |   31 |
+------+--------+--------+------+
3 rows in set (0.00 sec)
```

Table Department:

```
mysql> Select * from department ;
+---------+------+---------+
| dname   | dno  | tot_sal |
+---------+------+---------+
| Account |   21 |   35000 |
| HR      |   31 |   20000 |
+---------+------+---------+
2 rows in set (0.00 sec)
```

After insertion operations:

Table Employee:

```
mysql> Select * from employee;
+------+--------+--------+------+
| eid  | name   | salary | dno  |
+------+--------+--------+------+
|    1 | Juliet |  18000 |   21 |
|    2 | Romeo  |  17000 |   31 |
|    3 | Joseph |  20000 |   31 |
|    4 | Paul   |  18000 |   31 |
+------+--------+--------+------+
4 rows in set (0.00 sec)
```

Table Department:

```
mysql> Select * from department ;
+---------+------+---------+
| dname   | dno  | tot_sal |
+---------+------+---------+
| Account |   21 |   35000 |
| HR      |   31 |   38000 |
+---------+------+---------+
2 rows in set (0.00 sec)
```

c) After delete of record

SQL :

- Delimiter //
- Create trigger after_del
  after delete on employee
  for each row
  begin
  update department
  set tot_sal = tot_sal - old.salary
  where dno = old.dno ;
  end //
- Delimiter //
- Delete from employee where name = ‘Romeo’ ;

Output:

Before:

Table: Employee

```
mysql> Select * from employee;
+------+--------+--------+------+
| eid  | name   | salary | dno  |
+------+--------+--------+------+
|    1 | Juliet |  18000 |   21 |
|    2 | Romeo  |  17000 |   31 |
|    3 | Joseph |  20000 |   31 |
|    4 | Paul   |  18000 |   31 |
+------+--------+--------+------+
4 rows in set (0.00 sec)
```

Table: Depaetment

```
mysql> Select * from department ;
+---------+------+---------+
| dname   | dno  | tot_sal |
+---------+------+---------+
| Account |   21 |   35000 |
| HR      |   31 |   38000 |
+---------+------+---------+
2 rows in set (0.00 sec)
```

After delete operation:

Table: Employee

```
mysql> select * from employee;
    -> //
+------+--------+--------+------+
| eid  | name   | salary | dno  |
+------+--------+--------+------+
|    1 | Juliet |  18000 |   21 |
|    3 | Joseph |  20000 |   31 |
|    4 | Paul   |  18000 |   31 |
+------+--------+--------+------+
3 rows in set (0.00 sec)
```

Table: Department

```
mysql> select * from employee;
    -> //
+------+--------+--------+------+
| eid  | name   | salary | dno  |
+------+--------+--------+------+
|    1 | Juliet |  18000 |   21 |
|    3 | Joseph |  20000 |   31 |
|    4 | Paul   |  18000 |   31 |
+------+--------+--------+------+
3 rows in set (0.00 sec)
```

Note:

To show the procedure
$ SHOW PROCEDURE STATUS WHERE Db = 'databasename';

To delete the procedure
$ Drop Procedure/PROC YourProcedureName

You can view all the triggers created :
$ show triggers ;
Or
$ show triggers from database_name;

You can delete the triggers :
$ drop trigger trigger_name ;

Mistakes can be:

Inverted sign (“”)
Subtract sign (-)